Codebase Handover Report

First things first

Be sure to read my thesis (or at least scan it) since this is the refactored code that I used there, and I'm not going to retype what I did in my thesis here, just roughly what the code does. It's called "A field-data based BEP model: Towards a higher prediction accuracy of backwards erosion piping using statistical analysis on field performance data."

1 Outline

How the code roughly works is as follows.

1.1 Data

First you have your raw data. This comes from sources that are collected. These go in the `data` folder. A typical entry looks like this;

```
Nieuwkuijk_data = {
    'Country': 'The Netherlands',
    'Location': 'Nieuwkuijk',
    'Year': '1880',
    'Day+month': '29/12',
    'River/coast': 'River',
    'Levee/dam': 'Levee',
    'Survival/failure': 'Failure',
    'Source_Water_level_diff': {'type': 'range', 'min': 1.7, 'max': 3.74},
    'Source_Seepage_length': {'type': 'range', 'min': 27, 'max': 37}, #
        ↪ Buijs said max 32, Kanning said 37
    'Soil type (source)': 'Moderately fine sand',
    'Source': 'Buijs (2013) / Kanning (2012)',
    'Source_Aquifer_thickness': {'type': 'range','min': 38, 'max': 64},
    'Source_Blanket_thickness': {'type': 'range', 'min': 0, 'max': 2.8},
    'Source_Hydraulic_conductivity': {'type': 'range', 'min': 0.000198, '
        ↪ max': 0.000388},
    'Remarks': '(1) Failed after digging ditches on the outer side [Kanning
        ↪  (2012)]'
}
```

Which fields there are can be found in the `reference_columns.py` file in /src/utils. In case of numeric values it has to be in dictionaries. For point-values it is written as `{'type': 'point', 'value':` x`}` and `{'type': 'range', 'min':` x_{min}`, 'max':` x_{max}`}`. Once this is achieved, the database can be built.

1.2 Database

This is mainly done in the `Database.build_database()` function. This function first builds a dataframe, then fills it in with the entries from the data sources. Then it enriches the database with data it can find through BRO, GeoTOP and RWS water level data. These are collected in their own columns. From this raw data, the modelling values have to be quantified. This

is done through mappings (found in /src/utils) and conversion tables that can be found in the `Enrichment` object. The final missing values are then imputed through a chosen imputation algorithm, either it is filled in with the mean of the other values, or it is imputed via a Bayesian Network. After that, the useless columns are filtered out. Finally, the prediction values for existing models are calculated and filled in. This is nice for reporting. After this statistics can be calculated.

1.3 Statistics

The `Statistics` object has some plotting features that make plots and puts them in the /results/statistics folder. If you run `run_all_statistics` it does all of the statistics functions. There is also the `Model_comparison` function which is more focussed on existing models, and if you have one, your own model.

1.4 Modelling

Then finally you can build your (PyMC) Bayesian models. Both the linear and logistic models take the database automatically, so you just have to define models like the examples in `linear_models` and `logistic_models` and run them like in the example. There is `BayesianModel.train()` for one model and `BayesianModel.train_many()` for multiple models. These also have the `BayesianModel.plotall()` for getting outputs. These are stored under /results/**model_name**.

1.5 The main file

You run everything from your main file. An example is like this:

```
    from src.Database_object import Database
from src.Enrichment_object import Enrichment
from src.Statistics_object import Statistics

from linear_models.Bayesian_linear_object import BayesianModel
# from logistic_models.Bayesian_logistic_object import BayesianLogisticModel


from src.Database_object import Database
from src.Statistics_object import Statistics
from linear_models.Bayesian_linear_object import BayesianModel


if __name__ == "__main__":

    # Build database
    db = Database()
    db.build_database()

    # Run statistics
    stats = Statistics(db, selector="test", export_all=True)
    stats.run_all_statistics()

    # Train Bayesian linear model
    trainer = BayesianModel(db)
    trainer.train(
        model_name="LDaq_linear",
```

```
        redo=True,
        plot_all=True,
        export_all=True,
    )

    # Final dataframe if needed
    df = db.get_dataframe
```

So very classic with imports. You need the `if __name__ == "main"` wrapper if you want to run PyMC models, I don't know why exactly but there you go. You need to instantiate a class, and then you can call the separate methods. Usually you don't call methods starting with an underscore (_) since they are helper methods and be sure to play around with the all the different methods.

1.6 Now it's your turn :)

This is a VERY short introduction into my code. Below is some stuff that I generated with ChatGPT, maybe it's useful for you, maybe it's not but it does give an overview of all the classes and methods. Now this code is definitely not finished, be sure to use it as a base, or take what is useful for you with your own code. There are still some bugs in there;

- There is somewhere in the code the `Imputation_unit`, that is old and should be replaced by `Enrichment` since I merged them.
- Symbolic regression is not updated, and does not really work with the code right now. It's a class, but functionally unchanged from my old workflow.
- There is some hardcoded stuff maybe left in.
- The `selector` functionality (I wanted at first to have a way to have multiple versions of the database, for example if you want to take the mean in every range, then the max and then compare both) is still in there and kinda works, but not that well. If you want you can either remove the functionality and be sure to have the naming of the figures/files updated, or try to update it correctly for all methods.

If you have questions, be sure to send me an email at sjoerd.cnossen1@gmail.com. Don't use my student email if you find that somewhere since that should be deleted by now. Good luck!

2 Purpose

This report is meant to help the next developer understand how the repository works without having to reverse engineer it from scratch. It is written as a technical handover: what the main runtime flow is, how the internal data structures are shaped, what every class is responsible for, what each method does, and which parts of the project look stable versus fragile.

This report is based on static inspection of the code that was present in the uploaded repository. That means the descriptions below explain the code as written, including a few places where the implementation appears unfinished or internally inconsistent.

3 Practical reading order for a new maintainer

If someone new inherits this code, the fastest route to understanding it is:

1. read `main.py`;

2. read `Database.build_database()`;
3. read `Enrichment.enrich_entire_database()` and the four derived-feature methods;
4. read `finalize_modelling_values()` and `apply_extra_features()`;
5. read `linear_models/linear_models.py` and `BaseLinearBayesianModel.fit()`;
6. only then move to `Statistics`, `Model_comparison`, and `Symbolic_Regression`.

4 High-level architecture

At a high level the project is a piping-database and modelling workflow:

1. create a master dataframe from hard-coded failure cases and STOWA survival cases;
2. enrich rows with external geodata and water-level information;
3. convert source-specific values into final modelling columns;
4. engineer extra features and fill missing values;
5. generate descriptive statistics and plots;
6. train Bayesian regression or Bayesian logistic models;
7. compare the project's engineered database against legacy analytical models such as Sellmeijer, Bligh, Schmertmann, and SD.

The code is organised around a small set of core classes:

- `Database`: central in-memory dataframe and build pipeline.
- `Enrichment`: external data retrieval, source quantification, and imputation utilities.
- `Statistics`: exploratory plots and correlations.
- `BaseLinearBayesianModel` plus subclasses: Bayesian regression models for critical gradient.
- `BaseBayesianLogisticModel` plus subclasses: Bayesian classification models for failure probability.
- `Model_comparison`: evaluates analytical models already written into the dataframe.
- `Symbolic_Regression`: separate experimental symbolic-regression branch.

5 Main runtime entry point

The main executable script is `main.py`. Its runtime flow is straightforward:

1. instantiate `Database()`;
2. call `db.build_database()`;
3. instantiate `Statistics(db, selector="test", export_all=True)`;
4. call `stats.run_all_statistics()`;
5. instantiate `BayesianModel(db)`;
6. train the linear model `LDaq_linear`;
7. expose the final dataframe through `db.get_dataframe`.

So the intended story is: build data first, inspect it second, fit models third.

6 Data model and conventions

The repository revolves around a single pandas dataframe with a wide schema. The reference schema lives in `src/utils/reference_columns.py`. Important conventions:

- source values are often stored as dictionaries rather than plain numbers;
- point values use `{"type": "point", "value": x}`;
- uncertain values use `{"type": "range", "min": a, "max": b}`;
- final modelling columns are flattened to scalar numerics later in the pipeline;
- the main target used by regression models is `Global_gradient`;
- the main class label used by classification models is `Survival/failure`.

The three most important column groups are:

- **raw or source-specific columns**: `Source_...`, `BRO_...`, `GeoTOP_...`, raw payload columns like `BHRG_data`;
- **final modelling columns**: `Water_level_diff`, `Seepage_length`, `Aquifer_thickness`, `d10`, `Porosity`, and related engineered features;
- **prediction columns**: `H_c_sellmeijer`, `i_c_sellmeijer`, and similar columns for other analytical models.

7 Module-level utility and data-loader functions

7.1 `data/Failures.py`

This module defines `get_failure_df()`. It builds a dataframe of historical failure cases by filling the project reference schema row by row with hard-coded dictionaries. Its role is to provide the failure half of the database. It is not dynamic; the cases are manually curated.

7.2 `data/STOWA_data/STOWA_data.py`

This module defines `get_STOWA_df()`. It reads the Excel file `Wellen_csv.xlsx`, keeps a subset of columns, standardises naming into the project schema, converts dates into separate `Year` and `Day+month`, marks the source as STOWA, generates IDs, and discards rows where both date and water-level information are unrecoverable. This is the survival half of the database.

7.3 `src/utils/reference_columns.py`

This module defines the master schema and feature lists:

- `reference_columns`: full dataframe schema;
- `reference_df`: empty dataframe with that schema;
- `imputation_features`: columns targeted for missing-value filling;
- `modelling_features`: columns used in statistics and some models;

- `all_modelling_features`: a wider set including engineered transforms.

8 Class-by-class documentation

8.1 `Database`

Role. `Database` is the central container for the in-memory dataframe. It owns the full build pipeline and the derived-feature logic, and it exposes the dataframe to the rest of the project.

Constructor and basic dataframe operations

`__init__()` Creates an empty dataframe with the full reference schema. This means all later code can assume expected columns already exist.

`get_dataframe` Property returning a defensive copy of the internal dataframe. This is the main external read interface.

`add_column(name, values)` Adds a column if it does not already exist. It throws an error on duplicate names, so it is intended for schema extension rather than overwrite.

`update(col, mask, value)` Updates a column in rows selected by a callable mask. The mask function receives the dataframe and should return a boolean series.

`assign(col, func, mask=None)` Similar to pandas `assign`, but operates in place on the internal dataframe. If a mask is provided, only the subset is recomputed.

`export()` Writes the current dataframe to `src/BEP_database.pkl`. It first deletes any existing file at that path. This is the project's main cache/export mechanism.

Database build pipeline

`build_database(...)` This is the most important method in the repository. It tries to load a previously exported pickle first. If that fails or `force_rebuild=True`, it rebuilds from raw sources.

The method's intended stages are:

1. load exported dataframe if present;
2. otherwise call `get_failure_data()` and `get_STOWA_data()`;
3. concatenate the two sources;
4. normalise missing values and year format;
5. run `_pre_enrichment_filter()`;
6. optionally instantiate `Enrichment` and call `enrich_entire_database()`;
7. convert source-specific values into final modelling values through `finalize_modelling_values()`;
8. impute missing values;
9. compute derived features with `apply_extra_features()`;
10. run `_post_enrichment_filter()`;
11. coerce modelling columns to numeric;

12. compute analytical-model predictions;
13. export the final dataframe.

There are three very important handover notes here:

- the line `stowa_df = stowa_df[:20]` truncates the STOWA dataset for debugging, so a rebuild currently uses only the first 20 STOWA rows;
- the code calls `Imputation_unit(self)` even though no such class is defined anywhere in the repository, which means the full rebuild path appears broken unless that object exists outside the uploaded files;
- the cached-load path is therefore much safer than the full rebuild path in the repository as uploaded.

`get_failure_data()` Thin wrapper around `get_failure_df()`. It exists mainly to keep the build pipeline readable and centralised.

`get_STOWA_data()` Thin wrapper around `get_STOWA_df()`.

`_get_reference_columns()` Returns the reference schema from the utility module.

`_get_modelling_features()` Returns the main modelling-feature list.

`_get_all_modelling_features()` Returns the wider modelling-feature list that includes engineered features.

`_get_imputation_features()` Returns the list of features eligible for missing-value filling.

Feature engineering and filtering

`apply_extra_features()` Converts several scalar-or-dict columns into plain numeric arrays and computes extra geotechnical features:

- `Bedding_angle` from a logarithmic formula in `d50`;
- `Tan_Bedding_angle` and `Tan_Friction_angle`;
- `Uniformity_coefficient = d60/d10`;
- `Kozeny_Carman` permeability proxy;
- `Hydraulic_conductivity_KC`;
- `Permeability`;
- `Global_gradient = Water_level_diff / Seepage_length`.

It is the bridge between raw modelling variables and the engineered variables used later by statistics and models.

`_pre_enrichment_filter()` Applies early cleanup before external enrichment. It replaces zero blanket thickness with the smallest positive blanket thickness seen in the data, removes dams from the dataset, and removes survival rows without valid year information.

`_post_enrichment_filter(say_words=True)` Applies final filtering after enrichment. It drops rows without valid `Water_level_diff`, removes rows where that difference is non-positive, and removes STOWA survival rows that have no enriched `_data` payloads at all.

`finalize_modelling_values()` This method collapses source-specific columns into final scalar modelling columns. For each feature it looks for values in priority order `Source`, then `GeoTOP`, then `BRO`. Dict-style point and range values are converted to single numbers, usually by taking the mean of a range. This is the key harmonisation step that turns heterogeneous evidence into a single modelling-ready table.

Analytical model helpers

`sellmeijer(d70, D_aq, K, L)` Computes the critical head according to the Sellmeijer-style relation. It uses constant factors, permeability, aquifer thickness, and seepage length.

`sellmeijer_D_over_L(d70, D_over_L, K, L)` Variant of Sellmeijer where the thickness-to-length ratio is supplied directly.

`Bligh(d50, L)` Computes a Bligh-style critical head using piecewise coefficients based on grain size.

`schmertmann(d10, U, D_aq, L)` Computes a Schmertmann-style critical head using grain size, uniformity coefficient, aquifer thickness, and seepage length.

`schmertmann_D_over_L(d10, U, D_over_L, L)` Ratio-based Schmertmann variant.

`sd(d10, d50, D_aq, K, L)` Computes a critical head using the SD relation. This function is vectorised and performs several internal geotechnical transformations such as particle Reynolds number and critical shear surrogate calculations.

`icresult(L, D_aq)` Returns a simple empirical relation for critical head based on seepage length and aquifer thickness.

`fill_existing_model_predictions(overwrite=True, verbose=True)` Runs the analytical helper methods across the dataframe and writes prediction columns like `H_c_sellmeijer`, `i_c_sellmeijer`, and equivalents for other models. It also forces the prediction columns to numeric and masks non-positive results.

Export and helpers

`generate_kml_from_dataframe_manual_xml(...)` Writes a simple KML file from latitude and longitude columns. It manually assembles XML rather than using a GIS library. One likely bug: if `df is None`, the code tries to access `self._database`, but the class is `Database` itself and does not define `self._database`. This method probably needs a small fix before use.

`_to_scalar(value, how="mean")` Utility for converting supported value representations into a single float. It understands plain numbers, point dicts, and range dicts.

8.2 `Enrichment`

Role. `Enrichment` operates directly on the dataframe owned by a `Database` instance. It is responsible for external data retrieval, translation of raw external payloads into project-specific features, manual seepage-length insertion, and missing-value imputation.

`__init__(database)` Stores a reference to the parent database.

`df` Property returning the live dataframe from the parent database. Important detail: this is a property without a setter. Later methods that assign to `self.df` look incorrect and likely raise `AttributeError`.

Top-level orchestration

`enrich_entire_database(...)` The main row-by-row enrichment loop. It ensures the required raw payload columns exist, iterates over each row, performs selected enrichments, computes derived columns, and finally applies manually assigned seepage lengths. Internally the order is:

1. fetch BRO payloads;
2. fetch GeoTOP and REGIS payloads;
3. fetch Rijkswaterstaat water-level payloads;
4. compute `Water_level_diff`;
5. derive blanket thickness and soil parameters from BRO;
6. derive blanket thickness, aquifer thickness, and soil parameters from GeoTOP;
7. quantify source soil descriptions into numeric/range-valued columns;
8. apply manual seepage lengths by ID.

External raw fetches

`enrich_with_rws_waterlevel_data(...)` Looks up nearby Rijkswaterstaat stations through `ddlpy`, filters to relevant water-level measurement types, selects nearby stations by transformed distance, downloads time-series measurements in a window before the event date, and stores the raw payload in `RWS_Waterlevel_data`. It also caches results through the backup system.

`enrich_with_bro_data(idx, ...)` Fetches raw BRO borehole information for a row. It transforms WGS84 coordinates to Dutch RD coordinates, calls three lower-level BRO helpers, stores their payloads into `BHRG_data`, `BHRP_data`, and `BHRGT_data`, and caches the composite result.

`_get_bhrg_data(x, y, ...)` Performs a WMS `GetFeatureInfo` request against the geologic-borehole layer, then fetches the detailed XML object by BRO ID and parses location, depth, and a list of layer strings.

`_get_bhrp_data(x, y, ...)` Same pattern for another BRO product type. It returns location, depth, and a parsed layer list with component soil types.

`_get_bhrgt_data(x, y, ...)` Same pattern for geotechnical boreholes, again returning a parsed payload with distance, depth, location, and layer descriptions.

`enrich_with_geotop_regis_data(idx, max_depth=100)` Transforms coordinates to RD, fetches a simplified GeoTOP lithology profile and a simplified REGIS hydrostratigraphic profile, writes them into `GeoTOP_data` and `REGIS_data`, and caches the combined payload.

`_get_geotop_profile(x, y)` Reads a GeoTOP column and compresses consecutive equal lithologies into layer dictionaries with top, bottom, lithology, and thickness.

`_get_regis_profile(x, y, max_depth=100)` Reads a REGIS column, filters invalid layers, maps hydrostratigraphic codes to names, truncates total depth to `max_depth`, and returns simplified layer dictionaries with code, name, thickness, and horizontal/vertical conductivity.

Derived quantities

`get_waterlevel_differences(idx, force_recalculate=True)` Computes `Water_level_diff` strictly as `Water_level_from_NAP - Altitude`. It extracts a maximum water level from the RWS payload, backfills altitude from `sampled_hegihts.csv` if needed, and stores the result as a point dict.

`_write_unmapped_to_file(description, filepath)` Appends unseen soil descriptions to an "unmapped" text file so the mapping dictionaries can be extended later.

`get_porosity_by_description(description)` Maps a textual soil description to a porosity range dict. If the description is missing from the mapping table, it logs it to a text file.

`get_friction_angle_by_description(description)` Same idea for friction angle.

`get_hydraulic_conductivity_by_description(description)` Same idea for hydraulic conductivity.

`get_particle_sizes_by_description(description)` Maps a textual soil description to representative particle-size values `d10`, `d50`, `d60`, and `d70`.

`get_soil_thicknesses_BRO(idx, source_columns=...)` Scans BRO layer descriptions to estimate blanket thickness. Clay layers always count; thin sand layers may also count depending on thickness thresholds. The result is stored as a range dict.

`get_info_from_BRO_data(idx)` Chooses the nearest BRO payload among the available BRO object types, extracts soil descriptions, maps them to porosity, conductivity, friction angle, and particle-size estimates, and stores the derived values in `BRO_...` columns.

`get_info_from_GeoTOP(idx)` Interprets the simplified GeoTOP profile to identify a likely blanket and aquifer, stores blanket and aquifer thickness, stores the list of aquifer soils, and maps those soils to geotechnical property columns `GeoTOP_...`.

`source_quantifier(idx)` Converts the text field `Soil type (source)` into `Source_Porosity`, `Source_Hydraulic_conductivity`, `Source_Friction_angle`, and source particle sizes.

`apply_manual_seepage_lengths(overwrite=True, verbose=True)` Fills `Seepage_length` for a hard-coded set of IDs using a large lookup table. This is important because many cases depend on manually curated seepage lengths rather than computed values.

Imputation

`run_bayesian_network(...)` Performs Bayesian imputation over selected features. The method supports two modes: a simpler multivariate-normal model in scaled space and a more complicated copula-style mode. It saves the imputed dataframe and inference data to disk, then applies the imputed values back to the current dataframe.

`_apply_bayesian_imputation_to_df(name, imputation_features=None)` Loads a previously saved imputed dataframe and writes imputed values only into currently missing cells.

`run_mean_imputation(imputation_features=None, name='not_specified')` Computes simple column-mean imputation, saves the imputed dataframe, and then applies it back to the current dataframe.

`_apply_mean_imputation_to_df(name, imputation_features=None)` Loads a previously saved mean-imputed dataframe and only fills currently missing cells.

Important implementation warning. The four imputation methods assign to `self.df` even though `df` is defined as a property without a setter. As written, these methods likely fail or behave unexpectedly. This is one of the most important fixes to make before relying on these methods.

Backup-cache utilities

`_get_backup_path(enrichment_name, params)` Builds a deterministic pickle path for caching raw enrichment outputs. Parameter values are encoded into the filename.

`_row_backup_key(row)` Creates a row identifier from coordinates, year, and day/month.

`_empty_backup_df()` Returns an empty cache dataframe with the expected schema.

`_load_backup_df(enrichment_name, params)` Loads a cached dataframe if one exists, otherwise returns an empty cache table.

`_save_backup_df(backup_df, enrichment_name, params)` Writes a backup dataframe to disk.

`_find_backup_payload(backup_df, row)` Looks up a row in a cache table and returns the cached payload, a missing marker, or `None`.

`_upsert_backup_payload(backup_df, row, payload)` Replaces or appends one cache entry for the supplied row.

Value-format helpers

`_is_missing(v)` Custom missing-value detector that treats `None`, pandas NA, empty dicts, and empty lists as missing.

`_make_point(v)` Wraps a numeric value into the project's standard point-dict format.

`_make_range(min_v, max_v)` Wraps two numerics into the project's standard range-dict format.

`_ensure_value_dict(v)` Normalises an input into either a point dict, a range dict, or missing.

8.3 `Statistics`

Role. `Statistics` generates descriptive plots and correlation analysis for the modelling features.

`__init__(database, selector='test', export_all=True)` Pulls a dataframe copy from the database, chooses modelling features that actually exist, coerces them to numeric, loads an "imputed only" dataframe if available, and creates output directories.

`_labelname(name)` Maps raw column names to cleaner plot labels.

`_labelunits(name)` Maps raw column names to labels including units.

`_safe_filename(name)` Sanitises labels for filenames.

`_valid_series(col, positive_only=False)` Returns a cleaned numeric series from the dataframe.

`_class_series(col, klass, positive_only=False)` Returns a cleaned series for one class, usually `Survival` or `Failure`.

`_imputed_series(col, positive_only=False)` Returns the series of imputed values for a column.

`_feature_has_variation(arr)` Checks whether a variable has enough non-trivial spread for plotting or KDE estimation.

`_get_bins(col, data)` Generates sensible linear or logarithmic histogram bins depending on the feature.

`quickreport()` Prints a short summary of the number of failures and the percentage of modelling-feature cells that are filled.

`run_all_statistics(...)` Orchestrates the full statistics workflow and records which steps succeeded.

`histograms()` Produces stacked histograms for each feature, separating failures, survivals, and imputed values.

`KDE_plots()` Produces kernel-density plots for each feature where enough data and variation exist.

`scatter_plots()` Produces scatter plots of each modelling feature against `Global_gradient`, coloured by class.

`pair_plots()` Produces pair plots for the selected features. This is a broader exploratory visualisation of pairwise relations.

`correlation_matrix()` Computes and plots feature correlations, including significance filtering and CSV export of coefficient and p-value tables.

8.4 `ExplainingPower`

Role. This class appears to be intended as a feature-importance and explainability workbench.

`__init__(database, alphas=None)` Loads the dataframe, sets up plotting style, chooses predictor columns, maps the target `Survival/failure` to binary form, prepares result containers, and creates multivariate matrices. Inside the constructor there are local function definitions for methods such as L1 logistic, mutual information, random forest importance, permutation importance, elastic net, and stability selection.

Important handover note. Those analysis functions are nested *inside* `__init__` and are not exposed as class methods. As a result, the class as written only has two actual methods: the constructor and `fetch_results()`. The intended feature-importance workflow therefore looks incomplete in the uploaded code.

`fetch_results()` Returns the results dictionary accumulated by the class. In the current uploaded code, that dictionary is initialised but the nested analysis functions are not automatically run from outside the constructor, so the contents may remain empty unless the class was edited elsewhere.

8.5 `Model_comparison`

Role. This class compares analytical model outputs already stored in the dataframe against observed heads and gradients.

`__init__(database, selector='test', export_all=True, auto_fill_predictions=True)` Pulls a dataframe copy, sets output directories, and optionally asks the database to populate analytical-model prediction columns first.

Important handover note. The constructor expects `self._database.get_existing_model_registry()`, but `Database` in the uploaded code does not define this method. That means this class likely breaks unless that method exists in a missing file or older version.

`_safe_filename(name)` Sanitises labels for filenames.

`_labelname(name)` Maps raw names to cleaner labels.

`_json_safe(x)` Converts nested metric objects into JSON-serialisable types.

`_to_failure_binary(series)` Standardises a class indicator into 0/1 values.

`_regression_metrics(y_true, y_pred, k)` Computes RMSE, AIC, BIC, R^2, and adjusted R^2 for one predicted series.

`_classification_metrics_from_critical_head(y_true_cls, dH_obs, dH_crit)` Treats a model as predicting failure when observed head exceeds critical head, then computes confusion counts and classification metrics.

`_get_model_prediction_columns(model_name, spec)` Resolves the dataframe column names that hold one model's predicted critical head and predicted critical gradient.

`_build_model_summary_table()` Builds a summary table of model names, parameter counts, prediction columns, and whether those columns are present in the dataframe.

`compute_existing_model_metrics(export=True)` Computes regression metrics and threshold-based classification metrics for every analytical model in the registry and writes CSV/JSON summaries.

`plot_existing_model_predictions_vs_real(target='Water_level_diff', ...)` Produces observed-versus-predicted scatter plots for each model and each target.

`plot_ic_pairs_existing_models(lims=0.25, export=True)` Produces one combined plot comparing observed failure gradients against predicted critical gradients for all analytical models.

`plot_metric_bars(export=True)` Produces summary bar charts ranking models by RMSE and MCC.

`run_all_model_comparisons(...)` Orchestrates the full comparison workflow.

8.6 `BaseLinearBayesianModel`

Role. This is the reusable base class for Bayesian regression models that predict a continuous critical gradient. Subclasses provide feature selection, priors, and a mathematical core equation.

`__init__(...)` Stores configuration, loads the dataframe, validates that the subclass defines `priors` and `feature_cols`, and prepares output directories.

`_prepare_dataframe(df)` Hook for subclasses to add or transform columns before training. The base implementation returns the dataframe unchanged.

`get_training_dataframe()` Selects required columns, coerces them to numeric, drops rows with missing features or target, keeps only valid class labels, and returns the usable training table.

`_preprocess(df)` Min–max scales the features, converts `Survival/failure` to 0/1, and returns scaled predictors plus the continuous target.

`save_scaler()` Saves the fitted scaler to disk.

`load_scaler()` Loads the saved scaler from disk.

`core_equation(c, v)` Abstract hook to be implemented by subclasses. It defines the deterministic critical-gradient equation.

`_formula_latex()` Returns a display string for the subclass equation.

`fit(redo=True)` Builds and fits the PyMC model. The likelihood is asymmetric: survival cases are handled as censored observations and failure cases with a skew-normal likelihood. The method samples priors, optionally samples the posterior, adds posterior predictive draws, combines log-likelihood components, and saves inference data.

`_combine_loglik(idata, is_survival, is_failure, new_name='y_total')` Merges separate survival and failure log-likelihood arrays into one combined log-likelihood tensor for model-comparison criteria such as WAIC and LOO.

`_save_idata()` Writes inference data to NetCDF.

`_retrieve_idata()` Loads inference data from NetCDF.

`posterior_mean_prediction()` Returns the posterior mean predicted critical gradient for each observation.

`compute_logprobability(plot=True, export=False)` Aggregates posterior log-likelihood values and optionally plots their distribution.

`epic_metrics(export=True)` Computes regression metrics such as RMSE, AIC, BIC, R^2, adjusted R^2, failure-only RMSE, and, where possible, WAIC and LOO.

`classifier_plus_metrics(export=True)` Converts the continuous regression prediction into a failure classification by checking whether observed gradient exceeds predicted critical gradient, then computes classification metrics.

`show_formula(export=False)` Creates an image summarising the model equation and posterior coefficient summaries.

`plot_all_features(export=False, ...)` For each feature, varies that feature across a grid while keeping others fixed at their mean, then plots posterior predictive mean, credible interval, and prior curve.

`plot_residuals(export=False)` Plots posterior-mean residuals by observation index and class.

`plot_priors_posteriors(export=False)` Compares prior and posterior distributions for each coefficient.

`plot_rmse_distribution(export=False)` Uses posterior samples of `y` to visualise the distribution of RMSE.

`plot_arviz_stuff(export=False)` Produces ArviZ trace and energy plots.

`plotall(export_all=True)` Convenience wrapper that runs all plotting and metric methods.

`_json_safe(x)` Converts numpy-heavy metrics to JSON-safe values.

8.7 Linear Bayesian model subclasses

Each subclass only changes feature selection, priors, and the core deterministic equation.

`Dbl_exp` Uses only `Blanket_thickness`. The equation combines an exponential rise term and a linear blanket-thickness term plus an offset.

`Dbl_lin` Uses only `Blanket_thickness`. The equation is purely linear in blanket thickness.

`Daq_exp` Uses only `Aquifer_thickness`. The equation is an exponential function of aquifer thickness plus an offset.

`Daq_over_L_lin` Creates the derived feature `D_over_L = Aquifer_thickness / Seepage_length` inside `_prepare_dataframe()`, then fits a linear equation in that ratio.

`LDaq_exp` Uses `Seepage_length` and `Aquifer_thickness`. The equation is the sum of two exponential terms plus an offset.

`LDaq_linear` Uses `Seepage_length` and `Aquifer_thickness`. The equation is linear in both features. This is the model selected in `main.py`.

`LDblDaq_linear` Uses `Seepage_length`, `Blanket_thickness`, and `Aquifer_thickness`. The equation is linear in all three.

8.8 `BayesianModel`

Role. Simple trainer/manager for the linear Bayesian regression subclasses.

`__init__(...)` Stores global training settings and a pointer to the database.

`_get_registry()` Imports `linear_models.linear_models` and retrieves the model registry.

`available_models()` Returns the names of available model classes.

`train(model_name, redo=True, plot_all=True, export_all=True)` Instantiates the chosen subclass, fits it, and runs either the full plot suite or only metric exports.

`train_many(...)` Trains multiple model names in a loop.

8.9 `BaseBayesianLogisticModel`

Role. Reusable base class for Bayesian logistic models that predict failure probability directly rather than continuous critical gradient.

`__init__(...)` Stores configuration, loads the dataframe, validates subclass declarations, and prepares output directories and prediction containers.

`_prepare_dataframe(df)` Hook for subclass-specific feature engineering. Base implementation is identity.

`get_training_dataframe()` Extracts required feature and class columns, converts class labels to 0/1, drops incomplete rows, and returns the training table.

`_preprocess(df)` Min–max scales features, stores raw and scaled predictors, stores the binary target, and saves the scaler.

`save_scaler()` Saves the scaler.

`load_scaler()` Loads the scaler.

`core_equation(c, v)` Abstract hook returning the linear predictor on the logit scale.

`_mu_logit(v, c)` Thin wrapper around `core_equation()`.

`_formula_latex()` Returns a display formula string.

`fit(redo=True)` Builds the PyMC Bernoulli-logistic model, samples prior predictive draws, optionally fits the posterior, adds posterior predictive draws, saves inference data, and computes cached class predictions.

`_save_idata()` Writes inference data to NetCDF.

`_retrieve_idata()` Loads inference data from NetCDF.

`_calculate_all_predictions(threshold=None)` Computes prior and posterior mean failure probabilities and binary classifications at the chosen decision threshold.

`posterior_mean_probability()` Returns the posterior mean failure probability per observation.

`collect_metrics(export=True)` Computes accuracy, precision, recall, F1, MCC, ROC AUC, log loss, WAIC, and LOO, then optionally exports them.

`show_formula(export=True)` Produces an image summarising the model equation and posterior coefficient intervals.

`plot_arviz_stuff(export=True)` Produces trace and energy plots.

`plot_priors_posteriors_of_coefficients(export=True)` Compares prior and posterior coefficient distributions.

`plot_logistic_fit(export=True)` For each feature, plots observed binary outcomes and the fitted probability curve while other features are fixed at their mean.

`plot_contours(export=True, bonus_text=False)` For every pair of features, produces contour maps of posterior mean failure probability in scaled feature space.

`plot_confusion_matrix(threshold=None, export=True)` Visualises confusion-matrix percentages for posterior predictions, the observed class distribution, and the difference between the two.

`plotall(export_all=True)` Runs the full plotting and metric-export workflow.

`_json_safe(x)` Converts metrics to JSON-safe objects.

8.10 Logistic Bayesian model subclass

`HLU_log` This is the only registered logistic model in the uploaded code. It uses `Uniformity_coefficient`, `Water_level_diff`, and `Seepage_length`. The core logit is

$$\mu = c_1 U_c + c_2 \left(dH^{c_3} - L\right) + c_0.$$

So the model learns failure probability from a transformed head difference minus seepage length, plus a grain-uniformity contribution.

8.11 `BayesianLogisticTrainer`

Role. Manager/trainer for the logistic-model registry.

`__init__(...)` Stores configuration and the database reference.

`_get_registry()` Imports `logistic_models.logistic_models` and retrieves the registry.

`available_models()` Returns available logistic-model names.

`train(model_name, redo=True, plot_all=True, export_all=True)` Instantiates the chosen logistic subclass, fits it, and runs plots or metrics.

`train_many(...)` Trains multiple logistic models in sequence.

8.12 `Symbolic_Regression`

Role. Experimental branch for symbolic regression using `PySRRegressor`. It does not look tightly integrated with the rest of the repository and behaves more like a standalone script.

`__init__(df)` Stores a dataframe in `self.df`. The class inherits from `Database`, but it does not call the parent constructor, so it does not behave like a normal `Database` instance.

`get_equations(df, selection=None, failures_only=False, complexity=12, depth=6, save=True, title=...)` Runs symbolic regression on either all modelling features or a chosen subset. It scales predictors, maps `Survival/failure` to 0/1, trains PySR using a logistic-loss expression, prints discovered equations ranked by loss, and optionally writes them to text files.

`preprocess(df, feature_cols)` Converts selected feature columns to numeric, drops missing rows, and scales them to $[0, 1]$.

Important handover notes.

- This file contains substantial script-level side effects below the class definition. It reads `BEP_database_final.pkl`, preprocesses features, changes directories, and runs multiple symbolic-regression experiments when the module is imported.
- It imports modules like `reference_columns` and `symb_regress_func` using ad hoc path manipulation.

- Because of those side effects and path assumptions, this file looks experimental rather than production-safe.

9 Most important known issues and handover warnings

The following items are the places I would tell the next developer to inspect first.

Issue	Why it matters
`stowa_df = stowa_df[:20]` in `Database.build_database()`	Full rebuilds are currently debug-truncated to 20 STOWA rows, so any statistics or model training on a rebuilt database are not using the full dataset.
`Imputation_unit` is undefined	The core rebuild pipeline calls an imputation class that is not present in the uploaded repository. This likely breaks rebuilds after enrichment.
`Enrichment` imputation methods assign to `self.df`	`df` is a read-only property, so assigning to it is likely a bug. Bayesian and mean-imputation helpers should probably write to `self._database._df` directly.
`Model_comparison` expects `get_existing_model_registry()`	`Database` does not define this method in the uploaded code, so model comparison looks incomplete or dependent on a missing interface.
`generate_kml_from_dataframe_tributaries()` references `self._database`	That method does not exist on `Database`; small fix needed before using KML export.
Import-path inconsistency	Some files import from `src.`, some import plain module names like `Database_object` or `mappings`. This increases the risk of runtime import errors when moving working directories.
`ExplainingPower` nested functions live inside `__init__`	The intended feature-importance analyses are not exposed as real methods, so the class is incomplete as uploaded.
`Symbolic_Regression_Objects.py` runs code on import	Importing the module can trigger file reads and model runs. That is risky and should be moved behind a `if __name__ == "__main__"` block or into explicit functions.
Hard-coded file paths	Examples include `sampled_hegihts.csv` and one conductivity unmapped-description path pointing at a Windows desktop path. These will break portability.

10 Recommended maintenance strategy for the next developer

1. Stabilise the build path first: remove the STOWA truncation, replace `Imputation_unit` with the intended imputation entry point, and fix the `Enrichment.df` setter issue.
2. Standardise imports so every module can be run from the repository root without custom path hacks.
3. Decide which branches are production and which are experimental. The database builder, enrichment logic, statistics, and Bayesian regression path look like the real core. Symbolic regression and part of model comparison look more exploratory.
4. Add one integration test that rebuilds a tiny database and verifies that the main columns are present, numeric where expected, and export correctly.
5. Externalise hard-coded mappings and manual seepage-length tables if they are likely to grow.